

PRODUCT FEATURE GUIDE · 2026

Integration Tracker

End-to-end visibility for your integration landscape

Integration Tracker gives your team a single pane of glass to browse interfaces, investigate transaction logs, monitor success rates, and drill into failures — all without leaving the browser. Built for operations and integration teams who need fast answers without querying databases or parsing raw log files.

Dashboard

Interface Catalog

Logs Explorer

Transaction Tracking

Failure Analysis

Smart Caching

Integration Tracker · Internal Productivity Tool · June 2026

Table of Contents

#	SECTION	DESCRIPTION
1	Application Overview	Architecture, technology stack, and navigation
2	Dashboard	At-a-glance metrics, category breakdown, and activity charts
3	Integration Catalog	Browse, search, and manage all interface definitions
4	Interface Details	Per-interface specification, logs, filters, and success rate
5	Logs Explorer	Cross-interface log search with advanced filters
6	Event Type Model	Four-stage transaction lifecycle and status tracking
7	Filter & Search Capabilities	Full filter taxonomy available across screens
8	Caching & Sync	Per-session data caching and manual sync controls
9	API Integration	Endpoint mapping and mock/live toggle
10	Data Model	Interface and Log field reference
11	User Management	Authentication and administration

1 • Application Overview

Integration Tracker is a React 18 single-page application designed for integration operations teams. It connects to a REST API to fetch interface definitions and transaction logs, presenting them through a fast, filterable UI.

TECHNOLOGY STACK

- **Frontend:** React 18 + TypeScript
- **Build:** Vite
- **Styling:** Tailwind CSS
- **State management:** Redux Toolkit
- **Charts:** Recharts
- **Routing:** React Router v6

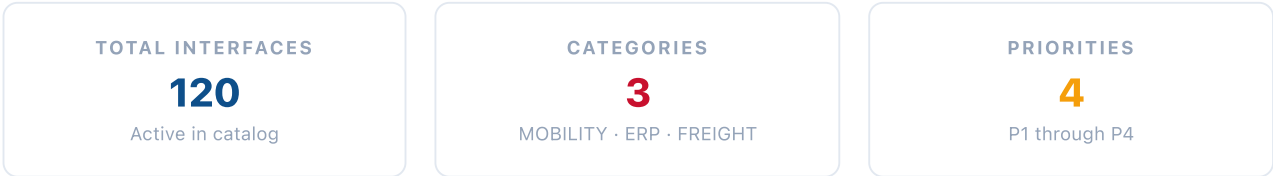
NAVIGATION STRUCTURE

- **/** — Dashboard
- **/integrations** — Interface Catalog
- **/projects/:id** — Interface Details
- **/logs** — Logs Explorer
- **/users** — User Management
- **/login** — Authentication

All routes except **/login** are protected — users must be authenticated to access any part of the application. Session state is stored in `sessionStorage` and cleared on logout.

2 • Dashboard

The Dashboard is the landing screen after login. It surfaces the most important operational metrics for the entire integration landscape without requiring any navigation.



Dashboard Widgets

Summary Stat Cards

Four top-level cards display total interfaces, active interfaces, category count, and priority breakdown. Values update automatically when interface data is refreshed.

Category Pie Chart

An interactive pie chart breaks down all interfaces by **ProjectOps** category (MOBILITY, ERP, FREIGHT). Hovering a slice shows the exact count and percentage.

Interface List Preview

A condensed list of interfaces with search and category filters. Each row links to the full Interface Details page. Responsive — adapts between card view (mobile) and table view (desktop).

Priority Breakdown

Visual representation of how interfaces are distributed across P1–P4 priority levels. Helps teams quickly understand the criticality profile of the integration portfolio.

3 • Integration Catalog

The Integration Catalog lists every interface in the system. It is the primary reference for understanding what integrations exist, what systems they connect, and how they are configured.



Search & Filter

- Free-text search across interface ID, name, source/target systems, and package name
- Filter by **ProjectOps** category (MOBILITY / ERP / FREIGHT)
- Filter by **Priority** (P1 / P2 / P3 / P4)
- Filter by **Status** (Active / Inactive)
- Instant results — no submit button required



Interface Card Fields

- **Interface ID** — unique identifier
- **Interface Name** — human-readable label
- **Source → Target** — system flow
- **Protocol** — source and target protocols
- **Priority** — P1–P4 badge
- **Category** — colour-coded chip
- **Frequency** — execution schedule

Clicking any interface card or row navigates directly to the **Interface Details** page for that interface, pre-loading its transaction logs.

4 • Interface Details

The Interface Details screen provides a complete operational view of a single interface — its specification, recent transaction logs, success rate, and inline filters. It is the primary troubleshooting surface for a specific interface.

Screen Sections



Specification Panel

Displays all metadata for the interface in a structured key-value layout:

- **Interface ID**
- **Interface Name**
- **Data Object**
- **Package Name**
- **Priority & Frequency**
- **Description**
- **Source Application**
- **Source Protocol**
- **Target Application**
- **Target Protocol**
- **Project / Category**
- **Status (Active / Inactive)**



Transaction Success Rate

The logs section header shows live metrics computed from the interface's transaction logs:

- **Total transactions** — unique Transaction IDs that have resolved
- **Succeeded** — transactions ending in a ● Success event
- **Failed** — transactions ending in a ● Failure event
- **Success rate %** — $succeeded \div (succeeded + failed) \times 100$

Success rate is calculated per **unique Transaction ID**, not by counting individual log entries. A transaction with Start → Info → Success counts as one successful transaction regardless of how many log lines it produced.



Log Filters (per-interface)

- **Keyword** — searches message, service, server
- **Transaction ID** — exact or partial match (monospace input with inline clear button)
- **Event type** — Start / Info / Success / Failure
- **Server** — filter by server name (shown when >1 server)
- **Time filter** — After / Before / Between with full date + time + seconds picker
- **Active chips** — removable colour-coded chips for each active filter



Log Table & Detail Panel

- Desktop: sortable table with columns — Event badge, Log ID, Transaction ID (truncated), Service, Message, Server, Timestamp
- Mobile: condensed card layout with the same fields
- **Click any row** to expand an inline detail panel showing all fields in a key-left / value-right layout
- Failure rows highlighted in red; expanded row highlighted in navy
- Results summary bar shows filtered count and Start · Info · ok · fail breakdown
- **Sync button** in the section header — forces a fresh API call regardless of cache

5 • Logs Explorer

The Logs Explorer aggregates recent transaction logs across *all* interfaces into a single searchable, filterable view. It is designed for cross-interface investigations, failure audits, and trend analysis.

Summary Stat Cards



Activity Bar Chart — Top 8 Interfaces

A stacked bar chart shows the top 8 most active interfaces by total log volume. Each bar is divided into four coloured segments matching the event type colour scheme. Hovering shows exact counts per segment.

• Start

• Info

• Success

• Failure



Advanced Filter Panel

Filters are arranged in two rows with active-filter chips below:

ROW 1 — SCOPE FILTERS

- **Interface** — multi-select dropdown with inline search; select one or many interfaces; supports "select all filtered"
- **Keyword** — searches message, service, server fields
- **Transaction ID** — monospace input with inline X clear; partial UUID match supported
- **Event type** — All / Start / Info / Success / Failure

ROW 2 — TIME FILTER

- **After** — logs created after a specific date + time + seconds
- **Before** — logs created before a specific date + time + seconds
- **Between** — logs within a date/time range (From → To pickers)

ACTIVE FILTER CHIPS

Each active filter renders as a removable colour-coded chip. Clicking X on a chip removes only that filter. A "Clear all" button resets the entire filter state.

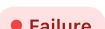


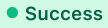
Paginated Log Table

- **20 records per page** with full pagination controls (first, prev, numbered pages with ellipsis, next, last)
- Page resets to 1 automatically whenever any filter changes
- **Results summary bar** — shows filtered count, total count, and Start · Info · ok · fail split for the current filter set
- Clicking any row expands an inline detail panel (key-left / value-right layout including payloads)
- Failure rows highlighted in red
- **Sync button** in the page header — animated spinner while loading

6 • Event Type Model

Every log entry carries one of four event types that represent the lifecycle stages of an integration transaction. A transaction is a group of log entries that share the same **TransactionID**.

EVENT	BADGE	MEANING	TYPICAL SEQUENCE POSITION
Start		Transaction initiated — inbound payload received and processing begun	Always first (required)
Info		Intermediate processing step — validation, transformation, or enrichment in progress	Second (optional, ~60% of transactions)
Success		Transaction completed — payload delivered and acknowledged by target system	Last (mutually exclusive with Failure)
Failure		Transaction failed — target rejected the payload or a processing error occurred	Last (mutually exclusive with Success)

Transaction lifecycle:  →  (optional) →  or .

The success rate is always computed as **successful transaction IDs ÷ resolved transaction IDs**, not by counting individual log entries.

7 • Filter & Search Capabilities

FILTER	AVAILABLE ON	BEHAVIOUR
Keyword search	Logs Explorer, Interface Details	Case-insensitive substring match across message, service name, server name
Transaction ID	Logs Explorer, Interface Details	Case-insensitive partial match — paste a full UUID or just a prefix
Interface (multi-select)	Logs Explorer only	Checkbox dropdown with inline search; select one, many, or all filtered interfaces
Event type	Logs Explorer, Interface Details	Exact match — All / Start / Info / Success / Failure
Server	Interface Details only	Shown when >1 server is present in the interface's logs; exact match
Time — After	Logs Explorer, Interface Details	Logs with CreatedDate after the selected datetime (date + time + seconds)

Time — Before	Logs Explorer, Interface Details	Logs with CreatedDate before the selected datetime
Time — Between	Logs Explorer, Interface Details	Logs within a From / To datetime range; both pickers shown simultaneously

All filters are **composable** — any combination can be applied simultaneously. The results bar and pagination update instantly on every keystroke or selection.

8 • Caching & Sync

To minimise unnecessary API calls and keep the UI fast, Integration Tracker implements a configurable per-session cache.



MAINTAIN_CACHE = true

- Each screen fetches its data **only once per login session**
- Returning to a screen re-uses the Redux store — no network call
- Interface Details caches logs **per interface ID** — navigating between different interfaces only fetches data for interfaces not yet loaded
- Cache is **cleared on logout**, ensuring the next session always starts fresh



Manual Sync Button

- A **Sync** button is always visible on Logs Explorer and Interface Details regardless of cache setting
- Clicking Sync **forces a fresh API call**, bypassing the cache, and overwrites stored data
- The button shows an animated spinner and "Syncing..." label while the request is in flight
- Useful when an incident is being investigated and the team needs the absolute latest data

9 • API Integration

All data is fetched via a configurable REST API. Three configuration flags in `config.ts` control the data source behaviour.

ENDPOINT	SCREEN	DESCRIPTION
GET <code>/getInterfaceList</code>	App-wide	Returns all interface definitions. Fetched once on layout mount.
GET <code>/getRecentLogs</code>	Logs Explorer	Returns recent logs across all interfaces. Fetched on first visit to Logs Explorer.
GET <code>/getLogsByInterfaceID/{id}</code>	Interface Details	Returns all logs for a specific interface. Fetched on first visit per interface ID.

All API responses must conform to the envelope structure: `{"response": {"results": [...]}}`. The client unwraps `response.results` before storing data in Redux.

CONFIGURATION FLAGS

- `USE_API` — `true` enables API mode; `false` uses bundled static JSON
- `USE MOCK` — when `true` alongside `USE_API`, the full API code path runs but all responses are served from local static files wrapped in the API envelope
- `MAINTAIN_CACHE` — controls per-session caching (see Section 8)
- `BASEURL` — base URL for all API calls (e.g. `http://localhost:8080`)

MOCK MODE BENEFITS

- Exercises the full API parsing and Redux flow without a live server
- Developers can work offline with production-representative data
- Switching to real APIs requires only setting `USE MOCK = false`
- Error handling, loading states, and Sync behaviour are identical in mock and real modes

10 • Data Model

Interface Fields

FIELD	TYPE	DESCRIPTION
<code>InterfaceID</code>	string	Unique identifier for the interface
<code>InterfaceName</code>	string	Human-readable name

DataObject	string	Business entity being transferred
PackageName	string	Integration package / middleware module
InterfacePriority	P1–P4	Business criticality level
InterfaceFrequency	string	Execution schedule (e.g. real-time, daily)
SourceApplication / Protocol	string	Origin system and communication protocol
TargetApplication / Protocol	string	Destination system and communication protocol
ProjectOps	MOBILITY / ERP / FREIGHT	Business domain category
IsActive	0 / 1	Whether the interface is currently active

Log Fields

FIELD	TYPE	DESCRIPTION
ID	7-digit number	Sequential unique log entry identifier
InterfaceID	string	Parent interface reference
TransactionID	UUID	Groups all log entries belonging to one transaction
EventType	Start / Info / Success / Failure	Stage in the transaction lifecycle
ErrorType	string / NULL	Error category for Failure events (e.g. TIMEOUT, VALIDATION_ERROR)
ServiceName	string	Fully-qualified middleware service that produced the log
LogMessage	string	Human-readable description of the event
ServerName	string	Server instance that processed the transaction
RequestPayload	JSON / NULL	Inbound payload (populated on Start events)
ResponsePayload	JSON / NULL	Target system response (populated on Success events)
ErrorPayload	JSON / NULL	Error details including code and retry count (Failure events)
IsAutoRetry	0 / 1	Whether the transaction was automatically retried after failure
CreatedDate	ISO 8601 UTC	Timestamp of the log entry to millisecond precision

11 • User Management



Authentication

- Login screen at `/login` — accepts username + password
- Session stored in `sessionStorage` — cleared automatically when the browser tab closes
- All routes behind **ProtectedRoute** — unauthenticated access redirects to login
- Logout clears session state *and* wipes the Redux data cache



User Management Page

- Dedicated page at `/users` under the Administration section
- Accessible from the sidebar navigation
- Reserved for future features: user roles, access permissions, team management, and audit logs